

CPSC-500-13

Nicholas Smith

Final Project - Omnichannel Shopping System

Group 5

Professor: Payman Janbakhsh

Group Members:

Nicholas Smith

Santiago Hernandez Riano

Lucas Santa

Ricardo Werchez De Paiva

Carlos Henrique Freitas

TABLE OF CONTENTS:

Professor: Payman Janbakhsh.....	1
1. Project Objective.....	3
2. System Overview.....	3
3. Dataset Requirements.....	4
4. Design Rationale.....	5
4.1. Schema and Relationship Design.....	5
4.2. Normalization and Data Integrity.....	5
4.3. Scalability and Maintainability.....	6
4.4. Support for Analytical Applications.....	6
5. DDL Features Implementation.....	7
5.1. Tables.....	7
5.2. Use of Views.....	7
5.2.1. Vw_orders_enriched.....	7
5.2.2. vw_product_sales_summary view.....	7
5.3. Use of XML.....	8
5.4. Use of JSON.....	8
5.5. Stored Procedures.....	9
5.6. Triggers.....	11
5.7. Indexes and Full-Text Search.....	11
6. Data Loading and DML.....	12
7. DQL: Query Design and Manipulation.....	13
7.1. Complex Queries with Joins and Aggregations.....	13
7.2. Window Functions and Ranking.....	13
7.3. XML and JSON Queries.....	14
7.4. Full-Text Search.....	14
8. Machine Learning - Ready Datasets.....	15
8.1. Customer Churn Classification Dataset.....	15
8.2. Order Value Regression Dataset.....	15
8.3. Product and Performance, and Inventory Dataset.....	16
9. Python Notebook and Analytics.....	17
9.1. Churn - Logistic Regression.....	17
9.2. Order Value -Linear Regression.....	18
10. Conclusion.....	19

1. Project Objective

This project focuses on designing and implementing a relational database system for a retail business that operates both physical stores and an online storefront. The system will support the management of customer activity across channels by recording purchases, tracking interactions, and storing relevant profile information. It will also manage store-level inventory data to show product availability in different locations. Additionally, this database will support promotional programs by linking customers to offers based on loyalty indicators and assessed risk levels. The system will store customer feedback, such as reviews and support requests, along with other interaction data. Overall, this platform aims to give the business a structured way to analyze customer behaviour, understand spending patterns, and make more informed decisions about customer engagement and targeted promotions.

2. System Overview

The database models key components of a modern omnichannel retail business using a set of interconnected tables governed by appropriate constraints. It stores customer master data and profile attributes, multiple addresses per customer, orders and order items, store-level inventory, product categories and product details, promotional campaigns, and customer interactions such as reviews or support requests.

The design emphasizes data accuracy, consistency, and traceability. Operational tables, such as orders, order_items, and inventory, support day-to-day processes, while supporting tables like customer_profiles, promotions, and interactions provide richer context for analytics, segmentation, and targeted promotions.

3. Dataset Requirements

The database is designed to support an omnichannel retail environment, which requires storing diverse but interconnected types of information. To meet the operational and analytical needs of the system, several categories of data were captured and organized into structured entities.

- **Customer data** is required to identify individuals who interact with the business across physical and online stores. This includes basic biographical information, such as their name and date of birth, contact information via phone and email, profile attributes, and behavioural indicators. Storing this data allows the company to maintain customer accounts, track loyalty, and support personalized engagement.
- **Address information** is necessary for order delivery, billing, and geographically based analytics. Since customers may use multiple locations for shipping and billing, the system must accommodate several address types per user.
- **Product data** describing the items available for purchase is another essential dataset. This includes product attributes, category assignments, pricing, and brand information. A hierarchical category structure supports detailed classification and analysis of products.
- **Store and inventory data** are required to represent stock availability across various retail locations. Due to the company operating both online and physical stores, inventory levels must be tracked per store to support accurate order processing and restocking decisions.
- **Order and order item data** are the primary transactional records in the system. These datasets capture completed purchases, the products involved, quantities, pricing details and the location where the transaction occurred. This information is essential for revenue tracking, customer behaviour analysis, and sales reporting.

4. Design Rationale

The database's schema contains 13 interconnected tables that follow normalization principles and have the correct primary and foreign keys to relate them.

4.1. Schema and Relationship Design

The structure of the database is based on accurately modelling the operational processes of an omnichannel retail system. Entities such as customers, products, orders, promotions, and interactions are separated into distinct tables to reflect their real-world roles to maintain conceptual clarity.

Key relationships were chosen to mirror business rules:

- A 1:1 relationship between customers and customer profiles avoids crowding the customers' table with optional or extended attributes.
- 1:M relationship between customers and addresses supports multiple billing or shipping locations without duplicating customer data.
- Hierarchical structure of categories allows products to be grouped across multiple levels, allowing flexible merchandising and reporting.
- M:N relationship between orders and products, connected through the order_items table, represents real-world transactions where multiple products appear in many orders.
- M:N relationships between orders and promotions support various types of promotional scenarios where some orders may be eligible for multiple offers.

4.2. Normalization and Data Integrity

Normalization principles guided the table design to reduce redundancy and maintain data consistency. The schema maintains Third Normal Form through several constraints:

- Data items such as customer addresses, product categories, and order line items are placed in their own tables to prevent repeated fields.
- Repeating groups, such as multiple products in an order or multiple promotions per order, are resolved using junction tables.
- Primary and foreign key constraints, ensuring valid references across tables.

- CHECK constraints, preventing invalid or empty values in key fields such as postal codes or country names.
- NOT NULL constraints for essential fields to prevent incomplete records
- Triggers (e.g. order status logs) to maintain change history and enforce business rules automatically.

The normalization and these constraints create a stable foundation where data remains consistent and logically structured in the omnichannel system.

4.3. Scalability and Maintainability

Scalability and maintainability were considered in the schema design and data type choices. Separating customer profiles, interactions, and promotions into their own tables allows those datasets to grow independently of the core transactional tables. The inventory table uses a composite primary key (store_id, product_id) to efficiently track stock levels across many store locations without duplicate rows.

The inclusion of JSON fields for flexible attributes allows the system to evolve without requiring frequent schema changes. Indexes on key columns such as customer_id, product_id, order_id, and date fields support efficient retrieval as data volume increases. These design decisions ensure that new features such as loyalty programs, additional interaction types, or more complex promotion rules can be added without reconstructing the entire schema.

4.4. Support for Analytical Applications

The schema is not only suitable for operational tasks but also designed to support analysis and decision-making. Customer behaviour can be studied by linking interactions, orders, and promotions. Sales performance can be analyzed through detailed order_items data aggregated at product, category, store, or time levels. Inventory patterns can be evaluated by examining stock levels across stores and linking them with order volume.

Promotion effectiveness can be assessed by analyzing the order_promotions table and comparing applied promotions with purchasing behaviour. Temporal analysis is supported through the order_status_logs table, which provides insight into fulfillment performance and lifecycle progression. Overall, the system provides a strong foundation for reporting, analytics, and business intelligence applications.

5. DDL Features Implementation

5.1. Tables

Each table is defined with explicit primary keys and foreign keys, appropriate data types, and CHECK constraints where useful. The design ensures that essential fields such as customer names, product names, order dates, and quantities cannot be null and that monetary values such as prices and totals are non-negative.

5.2. Use of Views

Views were included in the database to simplify access to commonly used information and reduce the complexity of analytical queries. The following views demonstrate how this approach supports reporting, improving usability, and helps.

5.2.1. Vw_orders_enriched

This vw_orders_enriched combines orders with customer and store information into a single, unified dataset. It eliminates the need to manually join multiple tables when reviewing order activity. The view improves reporting and customer service by providing quick access to complete order details. It also supports sales analysis by making it easier to compare store performance and understand customer purchasing patterns.

5.2.2. vw_product_sales_summary view

The vw_product_sales_summary view aggregates product performance metrics by combining data from products, categories, order_items, and orders. This view offers significant value for merchandising, pricing, and marketing teams. By summarizing sales volume and revenue, the organization can identify top-performing products, underperforming inventory, and category leaders. The ranking feature helps decision-makers quickly determine which items drive the most revenue within each product category, supporting decisions about restocking, discontinuing slow-moving products, and planning promotional strategies. Additionally, the inclusion of discount data enables the business to evaluate the effectiveness of markdowns and promotional tactics.

5.3. Use of XML

In a complex omnichannel retail environment, data is often stored in various formats, requiring specialized handling. While the core customer analysis was built on relational SQL tables, our project included a critical step of data cleaning and enrichment performed directly on semi-structured XML data contained within the products table. The following SQL command was used to correct and standardize product data by ensuring all relevant products contained an essential technical specification tag:

Figure 3. Code Excerpt of XML

```
UPDATE products
SET
    specs_xml = REPLACE(
        specs_xml,
        '</specs>',
        '<hdr_support>Yes</hdr_support></specs>'
    ),
    updated_at = CURRENT_TIMESTAMP
WHERE sku = 'LAB-MON-34-CURVED-240'
AND specs_xml NOT LIKE '%<hdr_support>%';
```

5.4. Use of JSON

The rules_json column in the promotions table uses the JSON data type to handle flexible, non-standardized promotion rules. This allows the table to maintain a structured format for key fields such as code, dates, and discount_value, while storing variable criteria such as product restrictions, customer segments, or usage limits without frequent schema changes. Using JSON enables efficient querying and analysis of promotion rules, supporting agile updates to marketing campaigns.

Figure 5.4 Code Snippet showing JSON usage

```
CREATE TABLE promotions (  
  -- ... core relational columns ...  
  discount_value DECIMAL(10,2) NOT NULL DEFAULT 0,  
  rules_json     JSON, -- <-- The key flexible column  
  is_active     BOOLEAN NOT NULL DEFAULT 1,  
  -- ...  
);
```

5.5. Stored Procedures

Two stored procedures were implemented to centralize key business logic and ensure that complex operations are executed consistently within the database layer.

- **sp_apply_promotion_to_order** handles the full workflow for applying a promotion to an order. The procedure validates the promotion code, confirms that the promotion is active and within its valid date range, and evaluates additional eligibility rules stored in JSON (such as minimum order value or allowed sales channels). It also prevents duplicate application of the same promotion, calculates the discount amount based on the promotion type, records the promotion in the junction table, and updates the order total accordingly. By encapsulating these steps, the procedure ensures that promotional logic is applied uniformly across the system.

Figure 5.5.1 Code Snippet Showing sp_apply_promotion_to_order procedure

```
CREATE PROCEDURE sp_apply_promotion_to_order (  
  IN p_order_id BIGINT,  
  IN p_promo_code VARCHAR(50)  
)  
BEGIN  
  DECLARE v_promotion_id INT;  
  DECLARE v_discount_type VARCHAR(20);
```

```

DECLARE v_discount_value DECIMAL(10,2);
DECLARE v_is_active      BOOLEAN;
DECLARE v_start_date     DATE;
DECLARE v_end_date       DATE;
DECLARE v_rules_json     JSON;
...
-- full procedure continues
END;

```

- **sp_update_inventory_for_order** adjusts store-level inventory for all products included in a given order. It verifies that the order exists, subtracts the purchased quantities from the corresponding inventory records, and returns the updated inventory rows. This procedure helps maintain accurate stock levels and provides immediate visibility into inventory changes following an order.

Figure 5.5.2 Code Snippet Showing sp_update_inventory_for_order procedure

```

CREATE PROCEDURE sp_update_inventory_for_order (
    IN p_order_id BIGINT
)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM orders WHERE order_id = p_order_id) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Order not found';
    END IF;
    ...
END;

```

Together, these procedures move business-critical calculations and validations into the database layer, improving consistency, reducing application-side complexity, and supporting transactional integrity.

5.6. Triggers

An AFTER UPDATE trigger on the orders table, called trg_orders_status_log, automatically records status changes. Whenever the status field of an order changes, the trigger

inserts a row into `order_status_logs` capturing the previous status, new status, timestamp, and a user marker. This ensures that the lifecycle of each order is auditable without manually inserting history records.

Figure 5.6. Code Snippet Showing `trg_orders_status_log`

```
CREATE TRIGGER trg_orders_status_log
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF NOT (OLD.status <=> NEW.status) THEN
        INSERT INTO order_status_logs (
            order_id,
            old_status,
            new_status,
            changed_at,
            changed_by
        )
        VALUES (
            NEW.order_id,
            OLD.status,
            NEW.status,
            CURRENT_TIMESTAMP,
            'system'
        );
    END IF;
END$$

DELIMITER ;
```

5.7. Indexes and Full-Text Search

The project includes a mix of standard and full-text indexes to support performance and search capabilities. Composite indexes, such as on `(customer_id, status, order_date)`, accelerate common queries such as retrieving a customer's order history filtered by status and time. An index on `order_items(product_id)` speeds up product-level aggregations.

A `FULLTEXT` index on products `(name, description)` enables natural-language queries against the product catalog. This allows SQL-based full-text search where users can search for items like "gaming monitor 34 inch curved" and receive results ordered by relevance score, mimicking basic e-commerce search behaviour.

6. Data Loading and DML

Bulk data for the main tables was imported from CSV files using MySQL Workbench's Table Data Import Wizard. The import followed a dependency-aware order: starting with dimension-style tables such as customers, customer_profiles, addresses, stores, categories, and products, then proceeding to inventory, orders, order_items, promotions, order_promotions, and interactions. This insert order avoided foreign key constraint violations.

On top of bulk imports, the DML scripts demonstrate INSERT, UPDATE, and DELETE operations along with JSON and XML manipulation. Examples include INSERTs for lab products with XML specs, lab promotions with JSON rule definitions, and lab interactions with JSON metadata. UPDATE statements show how to change customer statuses, update promotion rules using JSON_SET, modify interaction metadata, and update XML-in-TEXT fields using string functions and REPLACE. A controlled DELETE removes a specific lab interaction using a detailed WHERE clause and LIMIT 1.

7. DQL: Query Design and Manipulation.

The DQL portion of the project focuses on analytics-oriented SQL queries that join multiple tables, aggregate data, use window functions, and operate on XML and JSON fields. These queries demonstrate how the schema supports real business questions about revenue, product performance, promotion usage, and customer behaviour.

7.1. Complex Queries with Joins and Aggregations

Example queries include a monthly revenue report by channel and store type, which joins `orders`, `order_items`, and `stores`, grouping by year, month, channel, and store_type to compute monthly revenue, number of orders, and distinct customers. Another query aggregates sales by product brand by extracting the brand tag from `specs_xml` and grouping units sold and revenue by that brand.

Promotion usage is analyzed by joining `promotions`, `order_promotions`, and `orders`, and extracting JSON fields from `rules_json` to differentiate promotion types, minimum order values, and discount percentages. This allows the business to see which promotions drive the most orders and how much discount value is being granted.

7.2. Window Functions and Ranking

Window functions are used to compute running totals and rankings without collapsing rows. For example, a query computes monthly running revenue using `SUM(monthly_revenue) OVER (PARTITION BY channel, store_type ORDER BY year, month)`. The `vw_product_sales_summary` view uses `RANK() OVER (PARTITION BY category_name ORDER BY total_revenue DESC)` to rank products within each category based on revenue.

7.3. XML and JSON Queries

XML-in-TEXT fields are queried using string functions to extract specific tags. A common pattern in this project is to use nested SUBSTRING_INDEX calls to capture the value inside a tag, such as <brand>...</brand> from specs_xml for grouping and filtering.

JSON fields are accessed using functions such as JSON_EXTRACT and JSON_SET. In promotion analysis, JSON_EXTRACT is used to retrieve rule attributes like \$.type, \$.min_order_value, or \$.discount_pct from rules_json. Similar techniques are used on metadata_json in interactions to filter events by campaign, source, or device.

7.4. Full-Text Search

A dedicated query demonstrates FULLTEXT search on the products table using MATCH(name, description) AGAINST ('gaming monitor 34 inch curved' IN NATURAL LANGUAGE MODE). Leveraging the FULLTEXT index, this query returns relevance scores and orders results accordingly, showing how a SQL-only solution can provide a basic search experience for end users.

8. Machine Learning - Ready Datasets

The database is explicitly designed to generate machine learning–ready datasets directly from SQL. Three datasets were built for this project: a customer churn classification dataset, an order value regression dataset, and a product performance and inventory dataset suitable for clustering.

8.1. Customer Churn Classification Dataset

The customer churn dataset has one row per customer and a binary target variable, `churned_90d`. A customer is labelled as churned (1) if they have not placed an order in the last 90 days and as retained (0) otherwise. This definition captures recent inactivity and reflects a realistic churn concept for retail.

Features include first and last order dates, number of orders, total revenue, days since last order, counts of online versus in-store orders, and profile attributes such as `marketing_opt_in`, `preferred_language`, `preferred_channel`, `lifetime_value`, and `risk_score`. Together, these variables provide a rich representation of customer behaviour and potential value.

8.2. Order Value Regression Dataset

The order value regression dataset has one row per order and uses the numeric target `target_total_amount`, which is equal to the order's `total_amount`. Features include order status, channel, payment method, and time-based fields such as day of week and hour of day. `Store_type` and shipping geography provide additional context.

Line-level metrics derived from `order_items`—such as number of items, total quantity, total line amount, total discount amount, and number of promotions applied—capture the composition of each order. Extended versions of the dataset also include customer-level metrics such as `lifetime_value`, `risk_score`, and total number of orders.

8.3. Product and Performance, and Inventory Dataset

The product performance dataset has one row per product and is designed for future clustering or segmentation. Features include category and brand (extracted from specs_xml), technical attributes such as size and refresh rate, and performance metrics including units sold, net revenue, total discount amount, and number of orders.

Inventory-related features count how many stores carry each product and compute the average stock on hand. These variables could be used in clustering to group products into segments like high-revenue/low-stock, low-revenue/high-stock, or stable performers.

9. Python Notebook and Analytics

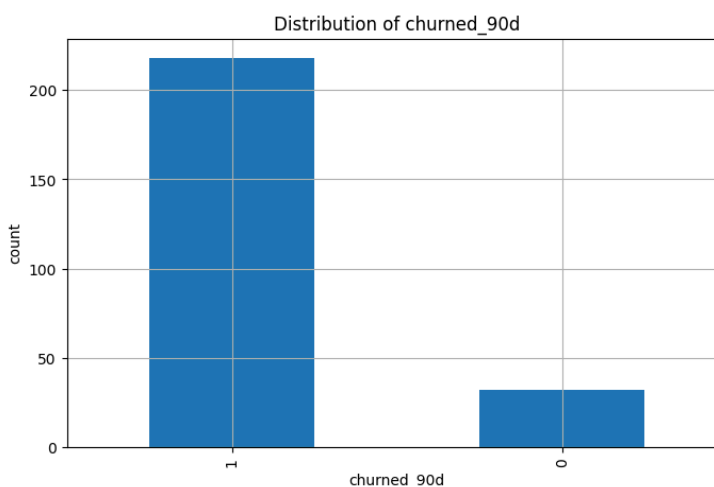
A Jupyter notebook was used to work with the exported CSV files, perform exploratory data analysis, and build simple machine learning models. The notebook demonstrates the end-to-end pipeline from SQL to analytics.

9.1. Churn - Logistic Regression

For the churn dataset, the notebook loads `customers_churn_dataset.csv` with pandas, inspects its shape and data types, and checks the distribution of the `churned_90d` variable. After selecting relevant numeric features and filling missing values where necessary, the data is split into training and test sets using a 70/30 split with stratification on the target.

A LogisticRegression model from scikit-learn is trained with an increased `max_iter` setting to ensure convergence. Model quality is evaluated using precision, recall, F1-score, and a confusion matrix. A simple boxplot (Figure 9.1) of the number of orders by churn label is used to visualize how purchase frequency relates to churn behaviour.

Figure 9.1

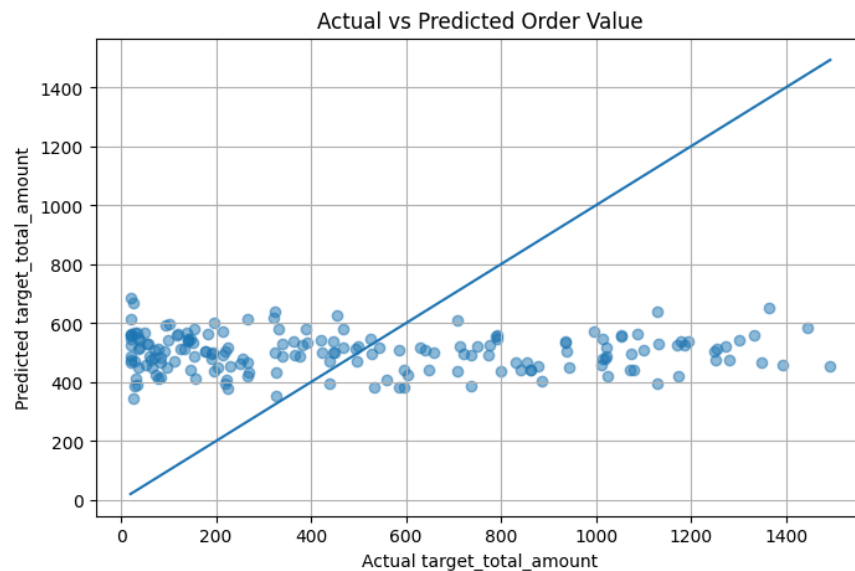


9.2. Order Value -Linear Regression

For the order value dataset, the notebook loads `orders_regression_dataset.csv`, verifies the distribution of `target_total_amount`, and selects numeric predictor variables such as `num_items`, `total_quantity`, `total_line_amount`, `total_discount_amount`, `num_promotions_applied`, `day_of_week`, and `hour_of_day`. The dataset is split into training and test sets using the same 70/30 split.

A LinearRegression model is trained and evaluated using RMSE and R-squared metrics. A scatter plot (Figure 9.2) of actual versus predicted `target_total_amount` with a diagonal reference line illustrates the model's fit and residual spread. Even though the models used are intentionally simple, they show how SQL-derived features can directly support predictive analytics.

Figure 9.2



10. Conclusion

The Omnichannel Shopping System database provides a complete and realistic example of how SQL can be used to support both operational processing and analytical workloads in a modern retail context. The schema enforces data integrity through constraints, triggers, and stored procedures while remaining flexible through the use of JSON and XML-in-TEXT columns.

Advanced SQL techniques, such as window functions, complex joins, and full-text search, enable rich analytical queries directly against the database. The creation of machine learning-ready datasets and their use in a Python notebook demonstrate how the database can power predictive models like churn and order value regression.

In future work, this platform could be extended with additional models such as product clustering, promotion uplift analysis, and recommendation systems. Dashboards and real-time reporting tools could be built on top of the existing views and queries to provide business stakeholders with up-to-date insights. Overall, the project illustrates an end-to-end data solution from schema design to analytics for the omni-channel business.